

AI Tool Prompt Engineering Report

Untal, Ray Emanuele P.
BS-CS 2C
16-607 21st Century IT Skills

I. AI Tool Overview

Name of AI Tool: Gemini

Purpose / Domain: Software Engineering / Web Development

Why you chose it: Aside from being my go-to AI tool whenever I code, I chose Gemini for its advanced capabilities to generate clean, functional code across multiple languages (like HTML, CSS, JS). It can handle both the logical back-end structures (such as API integration) and the front-end styling.

II. Original Prompt

My Initial Prompt:

Please generate me a simple weather dashboard using vanilla HTML/CSS/JS. The website must use any free weather APIs. Initially, the user is asked to input a city. The website must provide available options. When the city is not found, just turn to a page saying "No city is found" or whatever. Afterward, the dashboard should display the following information about the city:

Local time and date

Current temperature

Current weather and other related information

5 Days Forecast

Hourly Forecast Graph

For the design, implement a minimalist, modern, glassmorphism, and light/gradient theme.

Copy of the Output:

This weather dashboard uses the **WeatherAPI.com** (which offers a generous free tier including 3-day forecasts, hourly data, and location search). For the "Hourly Forecast Graph," I have used **Chart.js** to keep the implementation minimalist and clean.

Prerequisites

To make this work, you'll need an API Key.

1. Sign up for a free key at [WeatherAPI.com](https://weatherapi.com).
2. Replace 'YOUR_API_KEY' in the JavaScript section with your actual key.

1. HTML (**index.html**)

The structure focuses on a main container that toggles between the search state and the dashboard state.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

(full code provided in the Appendix section)

2. CSS (**style.css**)

This implements the **Glassmorphism** effect using **backdrop-filter** and semi-transparent gradients.

CSS

```
:root {
  --glass-bg: rgba(255, 255, 255, 0.2);
  --glass-border: rgba(255, 255, 255, 0.3);
  --text-color: #2d3436;
}
```

(full code provided in the Appendix section)

3. JavaScript (**script.js**)

The logic handles data fetching, the forecast loop, and the Chart.js rendering.

JavaScript

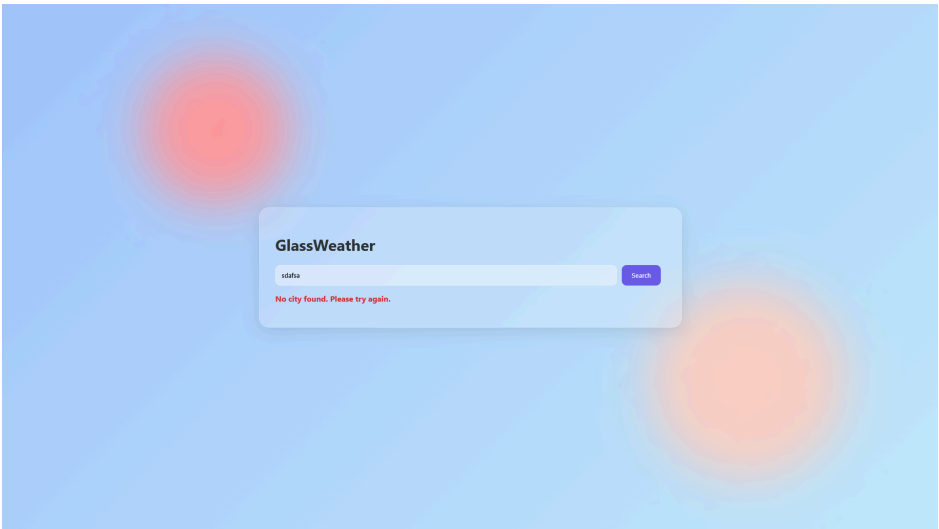
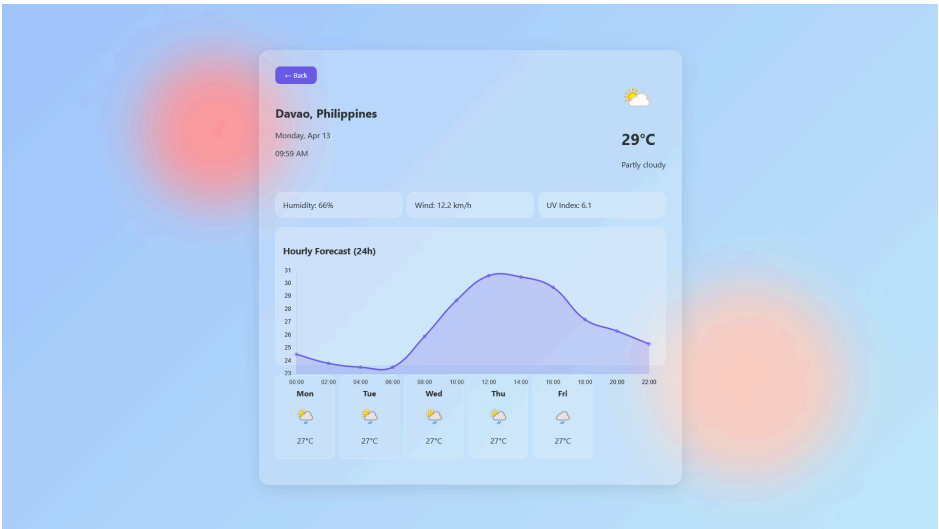
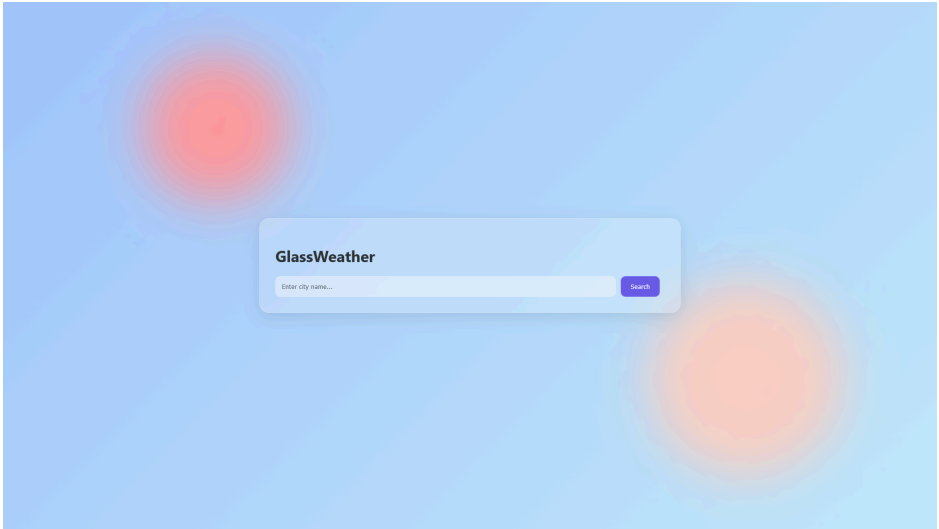
```
const API_KEY = 'YOUR_API_KEY'; // Replace with your WeatherAPI key
const searchBtn = document.getElementById('search-btn');
const cityInput = document.getElementById('city-input');
const errorMsg = document.getElementById('error-msg');
const searchSection = document.getElementById('search-section');
```

(full code provided in the Appendix section)

Key Features Included:

- **Minimalist Design:** Clean spacing and modern typography.
- **Glassmorphism:** Uses `backdrop-filter` and semi-transparent layers for a "frosted glass" look over a colorful gradient background.
- **Dynamic Background:** Large blurred circles behind the container provide depth.
- **Responsive Chart:** Uses Chart.js to visualize the 24-hour temperature swing.
- **Error Handling:** Displays a "No city found" message if the API returns an error.

Screenshots of the Generated Program



III. AI-Assisted Prompt

LLM Used:

Gemini

Improved Prompt:

Role: You are a Senior Frontend Developer specializing in modern UI/UX and Vanilla JavaScript.

Objective: Build a responsive, single-page Weather Dashboard using **HTML5**, **CSS3 (Custom Properties)**, and **Vanilla JavaScript (ES6+)**.

1. Data & API Requirements:

- **API:** Use the [OpenWeatherMap API](#) (Current Weather + 5-Day Forecast) or [WeatherAPI.com](#).
- **City Search:** Implement a search bar. Before fetching weather, use a **Geocoding API** or the weather API's "search/autocomplete" endpoint to provide a dropdown list of matching cities.
- **Error Handling:** If a city is not found or the API fails, display a dedicated "Error State" view (e.g., "City not found. Please try again") with a "Back to Search" button.

2. Dashboard Features:

- **Hero Section:** Display City Name, Local Time/Date (formatted via `Intl.DateTimeFormat`), Current Temperature (large font), Weather Description (e.g., "Partly Cloudy"), and a high-quality weather icon.
- **Detailed Metrics:** A grid showing Humidity, Wind Speed (km/h), UV Index, and Visibility.
- **5-Day Forecast:** A horizontal or vertical list showing the day of the week, weather icon, and High/Low temperatures.
- **Hourly Graph:** Use **Chart.js** (via CDN) to render a line chart of the next 24 hours of temperature data.

3. Visual Design (Glassmorphism & Light Theme):

- **Background:** A soft, animated linear gradient (e.g., `#a1c4fd` to `#c2e9fb`).
- **Glassmorphism:** Use `backdrop-filter: blur(15px);`, `background: rgba(255, 255, 255, 0.2);`, and a thin `1px solid rgba(255, 255, 255, 0.3);` border for cards.
- **Typography:** Use a modern sans-serif font (e.g., 'Inter' or 'Poppins').

- **Layout:** Use **CSS Grid** for the main dashboard and **Flexbox** for internal card alignment. Ensure it is fully responsive for mobile, tablet, and desktop.

4. Code Constraints:

- No frameworks (No React, Vue, or jQuery).
- Use `async/await` for all API calls.
- Organize the JS code into logical modules or a clean `App` object to handle State, UI Updates, and API Fetching separately.

Copy of the New Output

This is a sophisticated request for a Senior-level implementation. To keep this manageable and performant, I've structured the solution using a **Module Pattern**. We will use **WeatherAPI.com** as it provides the most comprehensive free-tier bundle for "Search/Autocomplete," "Current Weather," and "Forecast" in a single call.

1. The HTML Structure (`index.html`)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

(full code provided in the Appendix Section)

2. The CSS (Glassmorphism & Layout) (`style.css`)

CSS

```
:root {
  --bg-gradient: linear-gradient(-45deg, #a1c4fd, #c2e9fb, #e0c3fc,
#8ec5fc);
  --glass-bg: rgba(255, 255, 255, 0.2);
  --glass-border: 1px solid rgba(255, 255, 255, 0.3);
```

```
--text-primary: #2d3436;  
}
```

(full code provided in the Appendix Section)

3. The JavaScript (Logic & API) (**app.js**)

JavaScript

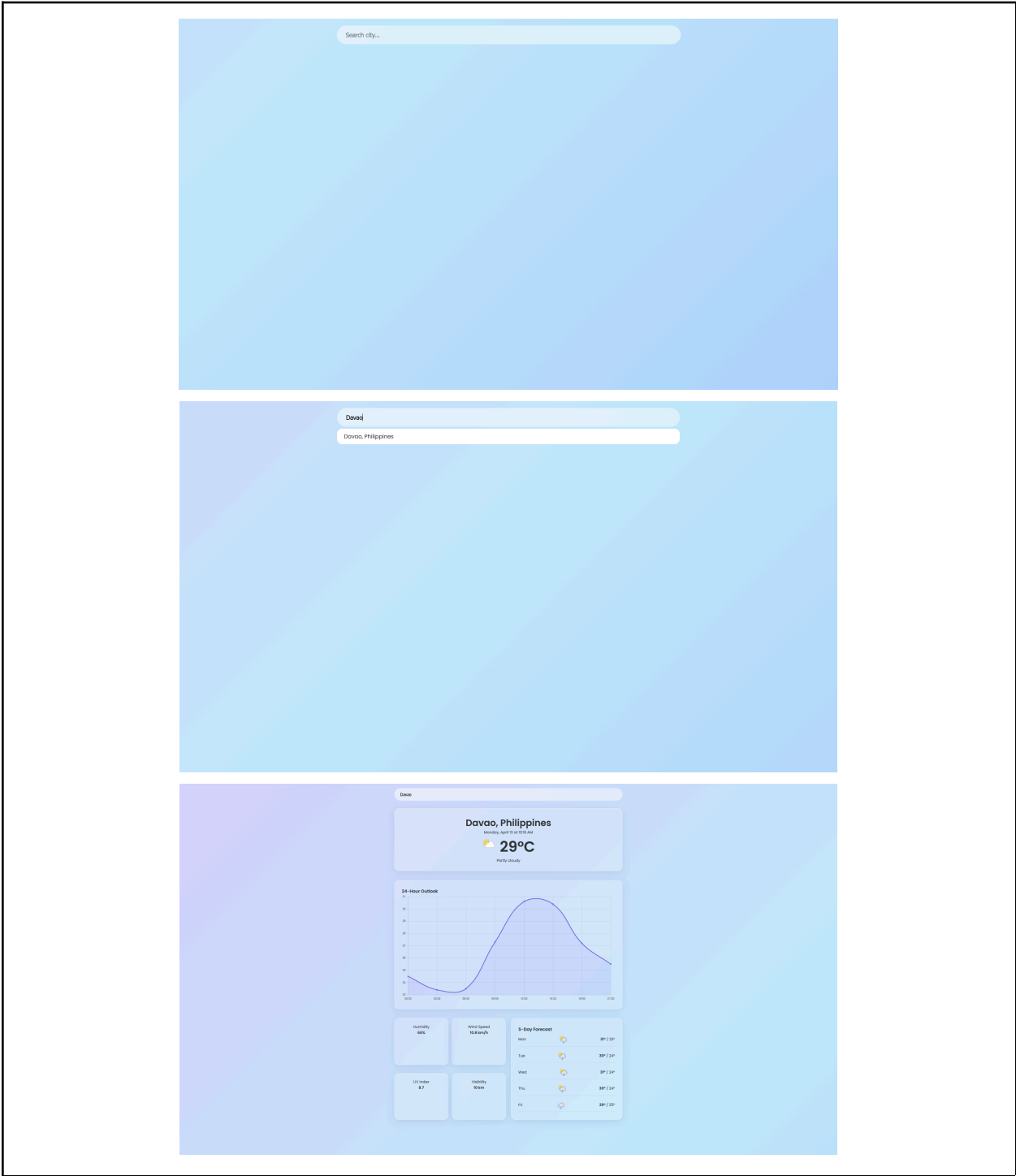
```
const API_KEY = 'YOUR_WEATHERAPI_KEY_HERE'; // Replace with actual  
key  
const BASE_URL = 'https://api.weatherapi.com/v1';
```

(full code provided in the Appendix section)

Why this works:

- **State Management:** The `App.state` object tracks the chart instance so it can be destroyed before re-rendering, preventing memory leaks and visual overlapping.
- **Performance:** The search input uses an "Autocomplete" flow, reducing broad API calls and improving user experience.
- **Visuals:** The `backdrop-filter` creates the glass effect, while the CSS animation keeps the UI feeling "alive."
- **Best Practices:** Uses `Intl.DateTimeFormat` for localizing time without heavy libraries like Moment.js.

Screenshots of the Generated Program



IV. Comparison and Analysis

For this laboratory assignment, we are tasked with creating AI-generated outputs for a specific domain using two separate prompts: one that I made myself and one generated and improved by AI. As a computer science student, I chose the domain of software engineering/web development, and settled on a simple weather-forecasting website.

Comparing the two prompts, the one I created and the one improved by AI, I can observe noticeable differences. My initial prompt consists of a single paragraph outlining my vision and project specifications. Meanwhile, the AI-improved prompt features role-playing, structured requirements, and specific technical constraints.

When I ran both prompts, the actual code structure for the HTML, CSS, and JS was surprisingly similar—both versions even utilized Chart.js for the graph. However, the most significant differences were in the User Experience (UX) and Visual Layout. In the output from my original prompt, the search functionality was very manual; I had to type a city and hit a "Search" button. If I made a typo, like typing "Davaos" instead of "Davao," it would simply trigger an error. In contrast, the AI-improved prompt generated an autocomplete/recommendation system. As I type, it provides a dropdown of valid cities, effectively preventing user error before it even occurs.

Furthermore, the visual polish differed greatly. The dashboard from my original prompt felt "compact" but had layout issues, such as the hourly graph bleeding outside its borders and sections overlapping. The AI-improved version, however, felt much "cleaner." It used a CSS Grid layout that handled spacing more professionally, ensuring the metrics and forecast sections were well-defined and didn't overlap.

The specific changes in the prompt that led to these better results were the Data & API Requirements and Layout Constraints. By explicitly asking the AI to use a "Geocoding API" for autocomplete and a "CSS Grid" for the main dashboard, I removed the "luck" factor. My original prompt was clear about the features, but the improved prompt was clear about the execution.

Through this activity, I learned that prompt engineering is essentially about narrowing the gap between "what I want" and "how to build it." As a developer, I realized that even if an AI knows how to code, it still needs specific architectural guidance to avoid common UI bugs, such as overlapping elements or poor spacing. This experience showed me that while AI is a powerful tool, its true potential is unlocked when you provide structured, detailed instructions that account for both logic and design.

Appendix

Code Generated by My Own Prompt

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>GlassWeather Dashboard</title>
    <link rel="stylesheet" href="style.css" />
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  </head>
  <body>
    <div class="background-circles">
      <div class="circle c1"></div>
      <div class="circle c2"></div>
    </div>

    <main class="glass-container">
      <section id="search-section">
        <h1 class="logo">GlassWeather</h1>
        <div class="search-box">
          <input
            type="text"
            id="city-input"
            placeholder="Enter city name..."
            autocomplete="off"
          />
          <button id="search-btn">Search</button>
          <div id="autocomplete-list" class="autocomplete-items"></div>
        </div>
        <p id="error-msg" class="hidden">No city found. Please try
again.</p>
      </section>
    </main>
  </body>
</html>
```

```
<section id="dashboard-section" class="hidden">
  <button id="back-btn">← Back</button>

  <div class="main-info">
    <div class="location-time">
      <h2 id="city-name">City Name</h2>
      <p id="local-date">Monday, 1 Jan</p>
      <p id="local-time">12:00 PM</p>
    </div>
    <div class="current-weather">
      <img id="weather-icon" src="" alt="Icon" />
      <h1 id="temp-display">24°C</h1>
      <p id="weather-desc">Cloudy</p>
    </div>
  </div>

  <div class="details-grid">
    <div class="glass-card">Humidity: <span
id="humidity">0%</span></div>
    <div class="glass-card">Wind: <span id="wind">0
km/h</span></div>
    <div class="glass-card">UV Index: <span id="uv">0</span></div>
  </div>

  <div class="chart-container glass-card">
    <h3>Hourly Forecast (24h)</h3>
    <canvas id="hourlyChart"></canvas>
  </div>

  <div class="forecast-container" id="forecast-list"></div>
</section>
</main>

<script src="script.js"></script>
</body>
</html>
```

script.js

```
const API_KEY = "b5411830226d4df3b8c15704261304"; // Replace with your
WeatherAPI key

const searchBtn = document.getElementById("search-btn");
const cityInput = document.getElementById("city-input");
const errorMsg = document.getElementById("error-msg");
const searchSection = document.getElementById("search-section");
const dashboardSection = document.getElementById("dashboard-section");
const backBtn = document.getElementById("back-btn");

let hourlyChart = null;

searchBtn.addEventListener("click", () => {
  const city = cityInput.value;
  if (city) fetchWeatherData(city);
});

backBtn.addEventListener("click", () => {
  dashboardSection.classList.add("hidden");
  searchSection.classList.remove("hidden");
  cityInput.value = "";
});

async function fetchWeatherData(city) {
  try {
    // We fetch 5 days (Free tier allows 3, but let's request 5 as per
    prompt)
    const response = await fetch(
      `https://api.weatherapi.com/v1/forecast.json?key=${API_KEY}&q=${city}&da
ys=5&aqi=no`,
    );

    if (!response.ok) throw new Error("City not found");

    const data = await response.json();
    renderDashboard(data);
    errorMsg.classList.add("hidden");
  } catch (error) {
    console.log(error);
  }
}
```

```

    } catch (err) {
      errorMsg.classList.remove("hidden");
      dashboardSection.classList.add("hidden");
    }
  }

function renderDashboard(data) {
  searchSection.classList.add("hidden");
  dashboardSection.classList.remove("hidden");

  // Basic Info
  document.getElementById("city-name").innerText =
    `${data.location.name}, ${data.location.country}`;
  const localDateTime = new Date(data.location.localtime);
  document.getElementById("local-date").innerText =
    localDateTime.toLocaleDateString("en-US", {
      weekday: "long",
      day: "numeric",
      month: "short",
    });
  document.getElementById("local-time").innerText =
    localDateTime.toLocaleTimeString([], {
      hour: "2-digit",
      minute: "2-digit",
    });

  // Current Weather
  document.getElementById("temp-display").innerText =
    `${Math.round(data.current.temp_c)}°C`;
  document.getElementById("weather-desc").innerText =
    data.current.condition.text;
  document.getElementById("weather-icon").src =
    `https:${data.current.condition.icon}`;

  // Details
  document.getElementById("humidity").innerText =
    `${data.current.humidity}%`;
  document.getElementById("wind").innerText = `${data.current.wind_kph}
km/h`;

```

```

document.getElementById("uv").innerText = data.current.uv;

// Forecast (Next 3-5 days depending on API plan)
const forecastList = document.getElementById("forecast-list");
forecastList.innerHTML = "";
data.forecast.forecastday.forEach((day) => {
  const div = document.createElement("div");
  div.className = "forecast-item glass-card";
  const date = new Date(day.date).toLocaleDateString("en-US", {
    weekday: "short",
  });
  div.innerHTML = `
    <p><strong>${date}</strong></p>
    
    <p>${Math.round(day.day.avgtemp_c)}°C</p>
  `;
  forecastList.appendChild(div);
});

// Hourly Graph
renderHourlyChart(data.forecast.forecastday[0].hour);
}

function renderHourlyChart(hours) {
  const ctx = document.getElementById("hourlyChart").getContext("2d");

  // Extract labels (time) and data (temp)
  const labels = hours
    .filter((_, i) => i % 2 === 0)
    .map((h) => h.time.split(" ")[1]);
  const temps = hours.filter((_, i) => i % 2 === 0).map((h) =>
h.temp_c);

  if (hourlyChart) hourlyChart.destroy();

  hourlyChart = new Chart(ctx, {
    type: "line",
    data: {
      labels: labels,

```

```

    datasets: [
      {
        label: "Temperature (°C)",
        data: temps,
        borderColor: "#6c5ce7",
        backgroundColor: "rgba(108, 92, 231, 0.2)",
        fill: true,
        tension: 0.4,
      },
    ],
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: { legend: { display: false } },
    scales: {
      y: { grid: { display: false }, ticks: { color: "#2d3436" } },
      x: { grid: { display: false }, ticks: { color: "#2d3436" } },
    },
  },
});
}

```

style.css

```

:root {
  --glass-bg: rgba(255, 255, 255, 0.2);
  --glass-border: rgba(255, 255, 255, 0.3);
  --text-color: #2d3436;
}

body {
  margin: 0;
  font-family: "Segoe UI", sans-serif;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

```



```
background: linear-gradient(135deg, #a1c4fd 0%, #c2e9fb 100%);
overflow: hidden;
color: var(--text-color);
}

/* Background decorative circles */
.background-circles .circle {
  position: absolute;
  border-radius: 50%;
  filter: blur(50px);
  z-index: -1;
}

.c1 {
  width: 300px;
  height: 300px;
  background: #ff9a9e;
  top: 10%;
  left: 15%;
}

.c2 {
  width: 400px;
  height: 400px;
  background: #fad0c4;
  bottom: 10%;
  right: 10%;
}

.glass-container {
  background: var(--glass-bg);
  backdrop-filter: blur(15px);
  -webkit-backdrop-filter: blur(15px);
  border: 1px solid var(--glass-border);
  border-radius: 20px;
  padding: 2rem;
  width: 90%;
  max-width: 800px;
  max-height: 90vh;
  overflow-y: auto;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.1);
}
```

```
}

.hidden {
  display: none !important;
}

/* Search Elements */
.search-box {
  position: relative;
  display: flex;
  gap: 10px;
  margin-top: 20px;
}

input {
  flex: 1;
  padding: 12px;
  border-radius: 10px;
  border: 1px solid var(--glass-border);
  background: rgba(255, 255, 255, 0.4);
  outline: none;
}

button {
  padding: 10px 20px;
  border-radius: 10px;
  border: none;
  background: #6c5ce7;
  color: white;
  cursor: pointer;
  transition: 0.3s;
}

button:hover {
  background: #a29bfe;
}

/* Dashboard Styling */
.main-info {
```

```
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 20px;
}

.temp-display {
  font-size: 3rem;
  margin: 0;
}

.glass-card {
  background: rgba(255, 255, 255, 0.25);
  border: 1px solid var(--glass-border);
  padding: 15px;
  border-radius: 15px;
  margin: 10px 0;
}

.details-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.forecast-container {
  display: flex;
  gap: 10px;
  overflow-x: auto;
  padding-bottom: 10px;
}

.forecast-item {
  min-width: 100px;
  text-align: center;
  padding: 10px;
  background: rgba(255, 255, 255, 0.15);
  border-radius: 12px;
}
```

```
.chart-container {  
  height: 250px;  
}  
  
#error-msg {  
  color: #d63031;  
  font-weight: bold;  
}
```

Code Generated by the AI-Improved Prompt

index.html

```
<!doctype html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width,  
initial-scale=1.0" />  
    <title>SkyGlass | Weather Dashboard</title>  
    <link rel="stylesheet" href="style.css" />  
    <link  
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&  
display=swap"  
    rel="stylesheet"  
  />  
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>  
  </head>  
  <body>  
    <div class="app-container">  
      <header class="search-section">  
        <div class="search-box">  
          <input
```

```
        type="text"
        id="cityInput"
        placeholder="Search city..."
        autocomplete="off"
    />
    <div id="autocomplete-list" class="autocomplete-items"></div>
</div>
</header>

<main id="dashboard" class="hidden">
    <section class="glass-card hero">
        <div class="hero-main">
            <h1 id="cityName">--</h1>
            <p id="localDateTime">--</p>
            <div class="temp-display">
                <img id="mainIcon" src="" alt="weather icon" />
                <span id="currentTemp">0</span>°C
            </div>
            <p id="weatherDesc" class="description">--</p>
        </div>
    </section>

    <section class="glass-card chart-container">
        <h3>24-Hour Outlook</h3>
        <canvas id="hourlyChart"></canvas>
    </section>

    <section class="metrics-grid">
        <div class="glass-card metric">
            <span>Humidity</span>
            <strong id="humidity">--%</strong>
        </div>
        <div class="glass-card metric">
            <span>Wind Speed</span>
            <strong id="windSpeed">-- km/h</strong>
        </div>
        <div class="glass-card metric">
            <span>UV Index</span>
            <strong id="uvIndex">--</strong>
        </div>
    </section>
</main>
```

```

    </div>
    <div class="glass-card metric">
      <span>Visibility</span>
      <strong id="visibility">-- km</strong>
    </div>
  </section>

  <section class="glass-card forecast-section">
    <h3>5-Day Forecast</h3>
    <div id="forecastContainer" class="forecast-list"></div>
  </section>
</main>

<section id="errorState" class="glass-card hidden">
  <h2>Oops! City not found.</h2>
  <p>Please check the spelling or try another location.</p>
  <button onclick="location.reload()">Back to Search</button>
</section>

<div id="loader" class="hidden">Scanning the skies...</div>
</div>

<script src="app.js" type="module"></script>
</body>
</html>

```

app.js

```

const API_KEY = "b5411830226d4df3b8c15704261304"; // Replace with actual
key
const BASE_URL = "https://api.weatherapi.com/v1";

const App = {
  state: {
    currentWeather: null,
    forecast: null,
    chartInstance: null,
  },

```

```

init() {
  this.cacheDOM();
  this.bindEvents();
},

cacheDOM() {
  this.cityInput = document.querySelector("#cityInput");
  this.autocompleteList =
document.querySelector("#autocomplete-list");
  this.dashboard = document.querySelector("#dashboard");
  this.errorState = document.querySelector("#errorState");
  this.loader = document.querySelector("#loader");
  this.chartCtx =
document.getElementById("hourlyChart").getContext("2d");
},

bindEvents() {
  this.cityInput.addEventListener("input", (e) =>
    this.handleSearch(e.target.value),
  );
  document.addEventListener(
    "click",
    () => (this.autocompleteList.innerHTML = ""),
  );
},

async handleSearch(query) {
  if (query.length < 3) return;
  try {
    const res = await fetch(
      `${BASE_URL}/search.json?key=${API_KEY}&q=${query}`,
    );
    const cities = await res.json();
    this.renderAutocomplete(cities);
  } catch (err) {
    console.error("Search failed", err);
  }
},

```

```

renderAutocomplete(cities) {
  this.autocompleteList.innerHTML = cities
    .map(
      (city) => `
        <div data-name="${city.name}">${city.name},
        ${city.country}</div>
      `,
    )
    .join("");

  this.autocompleteList.querySelectorAll("div").forEach((el) => {
    el.addEventListener("click", () => {
      this.fetchWeatherData(el.dataset.name);
      this.cityInput.value = el.dataset.name;
      this.autocompleteList.innerHTML = "";
    });
  });
},

async fetchWeatherData(city) {
  this.toggleLoader(true);
  try {
    const res = await fetch(
      `${BASE_URL}/forecast.json?key=${API_KEY}&q=${city}&days=5&aqi=no`,
    );
    if (!res.ok) throw new Error("City not found");
    const data = await res.json();
    this.updateUI(data);
  } catch (err) {
    this.showError();
  } finally {
    this.toggleLoader(false);
  }
},

updateUI(data) {
  this.errorState.classList.add("hidden");
}

```



```

this.dashboard.classList.remove("hidden");

// Hero Update
document.getElementById("cityName").textContent =
  `${data.location.name}, ${data.location.country}`;
document.getElementById("localDateTime").textContent =
  new Intl.DateTimeFormat("en-US", {
    weekday: "long",
    month: "long",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(data.location.localtime));

document.getElementById("currentTemp").textContent = Math.round(
  data.current.temp_c,
);
document.getElementById("weatherDesc").textContent =
  data.current.condition.text;
document.getElementById("mainIcon").src =
data.current.condition.icon;

// Metrics
document.getElementById("humidity").textContent =
  `${data.current.humidity}%`;
document.getElementById("windSpeed").textContent =
  `${data.current.wind_kph} km/h`;
document.getElementById("uvIndex").textContent = data.current.uv;
document.getElementById("visibility").textContent =
  `${data.current.vis_km} km`;

this.renderForecast(data.forecast.forecastday);
this.renderChart(data.forecast.forecastday[0].hour);
},

renderForecast(days) {
  const container = document.getElementById("forecastContainer");
  container.innerHTML = days
    .map((day) => {

```

```

        const date = new Date(day.date).toLocaleDateString("en-US", {
            weekday: "short",
        });
        return `
            <div class="forecast-item">
                <span>${date}</span>
                
<span><strong>${Math.round(day.day.maxtemp_c)}°</strong> /
${Math.round(day.day.mintemp_c)}°</span>
                </div>
            `;
        })
        .join("");
    },

    renderChart(hours) {
        if (this.state.chartInstance) this.state.chartInstance.destroy();

        const labels = hours
            .filter((_, i) => i % 3 === 0)
            .map((h) => h.time.split(" ")[1]);
        const temps = hours.filter((_, i) => i % 3 === 0).map((h) =>
h.temp_c);

        this.state.chartInstance = new Chart(this.chartCtx, {
            type: "line",
            data: {
                labels,
                datasets: [
                    {
                        label: "Temp °C",
                        data: temps,
                        borderColor: "#6c5ce7",
                        backgroundColor: "rgba(108, 92, 231, 0.1)",
                        fill: true,
                        tension: 0.4,
                    },
                ],
            },
        ],
    },

```

```

    },
    options: { responsive: true, plugins: { legend: { display: false }
} },
    });
},

showError() {
    this.dashboard.classList.add("hidden");
    this.errorState.classList.remove("hidden");
},

toggleLoader(show) {
    this.loader.classList.toggle("hidden", !show);
},
};

App.init();

```

style.css

```

:root {
    --bg-gradient: linear-gradient(-45deg, #a1c4fd, #c2e9fb, #e0c3fc,
#8ec5fc);
    --glass-bg: rgba(255, 255, 255, 0.2);
    --glass-border: 1px solid rgba(255, 255, 255, 0.3);
    --text-primary: #2d3436;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: "Poppins", sans-serif;
    background: var(--bg-gradient);
    background-size: 400% 400%;

```

```

    animation: gradientBG 15s ease infinite;
    min-height: 100vh;
    color: var(--text-primary);
    display: flex;
    justify-content: center;
    padding: 20px;
}

@keyframes gradientBG {
    0% {
        background-position: 0% 50%;
    }
    50% {
        background-position: 100% 50%;
    }
    100% {
        background-position: 0% 50%;
    }
}

.app-container {
    width: 100%;
    max-width: 1000px;
}

/* Glassmorphism utility */
.glass-card {
    background: var(--glass-bg);
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);
    border: var(--glass-border);
    border-radius: 20px;
    padding: 2rem;
    box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.1);
    margin-bottom: 20px;
}

/* Search Box */
.search-section {

```

```
    position: relative;
    margin-bottom: 2rem;
}
#cityInput {
    width: 100%;
    padding: 15px 25px;
    border-radius: 30px;
    border: var(--glass-border);
    background: rgba(255, 255, 255, 0.5);
    font-size: 1.1rem;
    outline: none;
}

.autocomplete-items {
    position: absolute;
    top: 60px;
    left: 0;
    right: 0;
    z-index: 99;
    background: white;
    border-radius: 15px;
    overflow: hidden;
}

.autocomplete-items div {
    padding: 10px 20px;
    cursor: pointer;
}

.autocomplete-items div:hover {
    background-color: #f1f1f1;
}

/* Dashboard Layout */
#dashboard {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
```

```
.hero {
  grid-column: 1 / -1;
  text-align: center;
}

.hero h1 {
  font-size: 2.5rem;
}

.temp-display {
  font-size: 4rem;
  font-weight: 600;
}

.chart-container {
  grid-column: 1 / -1;
}

.metrics-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 15px;
}

.metric {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 1.5rem;
}

.forecast-list {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.forecast-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
padding: 10px 0;
border-bottom: 1px solid rgba(0, 0, 0, 0.05);
}

.hidden {
  display: none !important;
}

@media (max-width: 768px) {
  #dashboard {
    grid-template-columns: 1fr;
  }
}
```